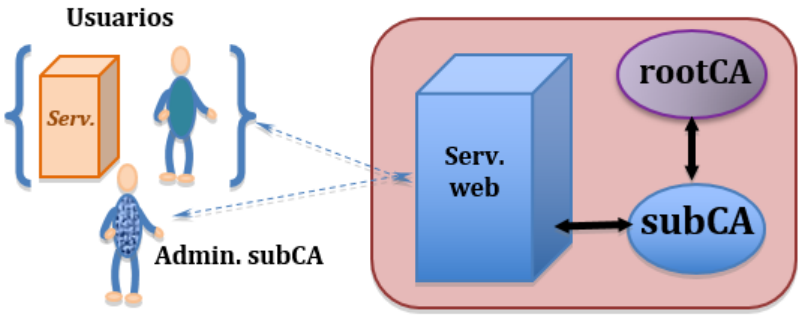
Práctica : PKI (1ª fase)

Objetivo

Consolidar los conceptos teóricos relacionados con certificados digitales. La parte fundamental de esta práctica será poner en funcionamiento[1] una PKI simple compuesta por una autoridad raíz de certificación (*rootCA*) y una autoridad de certificación subordinada (*subCA*) que emita los certificados requeridos en una red privada (*on-premises* CA) [2].

Tareas

1. Poner en marcha una autoridad raíz de certificación (*rootCA*) que emita/revoque/verifique certificados para CA subordinadas .
2. Poner en marcha una autoridad de certificación subordinada (*subCA*) que emita/revoque/verifique certificados para usuarios (personas y servicios).
3. Diseñar una interfaz web para la autoridad de certificación subordinada, con la descripción de todas las funcionalidades necesarias para interactuar con ella.



Documentación a entregar:

- Elaborar un **guión detallado** con todos los **pasos** necesarios **comentados** [*scripts*], incluyendo ficheros de configuración, capturas de pantalla de las ejecuciones de los comandos (mostrando el *prompt* del sistema personalizado y preferiblemente con fondo blanco) que indique como :
  - Crear la autoridad de certificación raíz

[1] Recomendado utilizar la máquina virtual proporcionada **personalizada** según las indicaciones

[2] En una segunda fase se hará uso de estos certificados con la puesta en marcha de sitios web seguros.

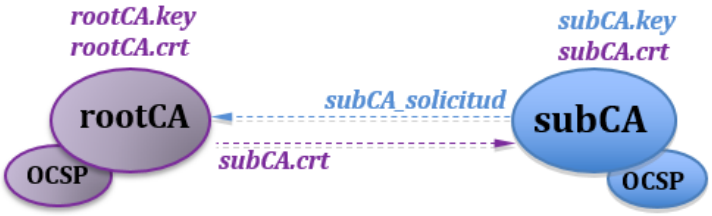
    - Crear la autoridad de certificación subordinada
    - Generar certificados para usuarios (personas y sitios web)
    - Revocar certificados
    - Verificar *status* de los certificados (CRL, OCSP)
    - ...
- Fichero comprimido con el directorio que contiene la **PKI** tras su completa puesta en marcha
- **Diseño de una aplicación web** que sirva de interfaz/*frontend* con la *subCA* (no es necesario para la *rootCA*)
    - Permitir acceso a:
        - Usuarios para solicitar, obtener y verificar certificados
        - Administrador de la *subCA* para gestionar todas las tareas de dicha CA (acceso restringido).
    - No hay que implementarla, pero hay que relacionar cada elemento de la interfaz de la aplicación web con su operación/función en el guión del punto anterior.
- Contabilizar el tiempo invertido

Ejercicios extra

- Probar opciones adicionales de `openssl`
- Certificados para sitios web con extensión SAN
- ...

Descripción del servicio ofrecido por una CA

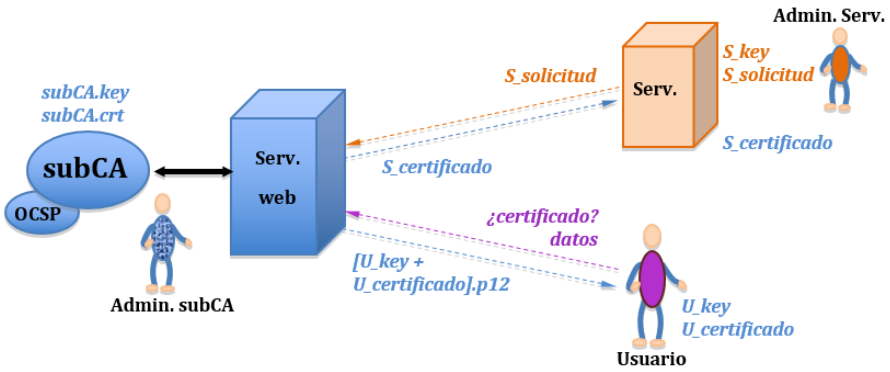
- Una CA raíz genera certificados para una CA subordinada:
  - Emitirá un certificado para la CA subordinada, válido solo para expedir certificados de usuarios



- Una CA subordinada puede generar dos tipos de certificados:
  - Personal:

    El usuario solicita un certificado para él y/o su dirección de correo electrónico. Proporciona sus **datos personales** que la CA subordinada verificará; si supera esta verificación remitirá por correo al usuario un fichero protegido con contraseña que contendrá el par $<$*clave privada, certificado*$>$
  - Para un servicio:

    El administrador de un servicio solicita un certificado. Proporciona a la CA subordinada una **solicitud de certificado** (que incluye una clave pública y datos adicionales sobre el servicio). La *subCA* verificará los datos y firmará el certificado que hará llegar al solicitante



- La emisión de certificados no es automática. En esta práctica, el administrador de la CA asumirá el papel de la Autoridad de Registro (RA), verificando los datos aportados por los solicitantes, y concediendo o denegando la emisión del certificado.
- Publicar su certificado, hacerlo accesible a todos los clientes de la red
- Revocar certificados
- Gestionar los números de serie de los certificados emitidos y revocados
- Gestionar un servidor OCSP para verificar online certificados emitidos por la CA
- Gestionar la lista de certificados revocados (CRL), mantenida y distribuida desde la CA

SOPORTE: Crear una infraestructura de clave pública (PKI)

OpenSSL es una colección de bibliotecas de funciones y comandos para criptografía y comunicaciones bajo el protocolo SSL/TLS. Proporciona todos los comandos necesarios para poner en marcha una PKI[3]. En esta actividad se utilizará una aplicación sencilla (`ca`) disponible en OpenSSL que facilita la gestión de una base de datos para los certificados emitidos y revocados por la CA, y que trabaja con un fichero de configuración donde se recoge información para emitir y revocar certificados.

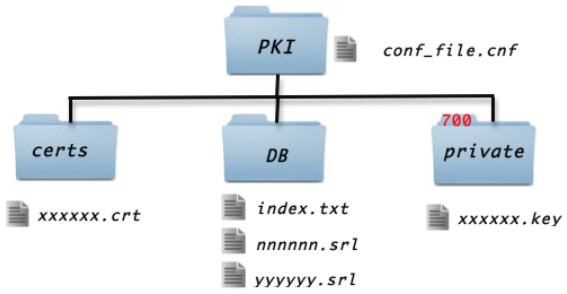
Creación de una Autoridad de Certificación

1. Crear la estructura de directorios de la *rootCA* (y de la *subCA*).

   Por ejemplo:

   En un directorio principal se crean tres subdirectorios[4] que contendrán:

   - Certificados emitidos por la CA
   - Ficheros con números de serie para certificados emitidos y CRLs; y el fichero índice para la base de datos
   - Clave privada de la CA (recomendable directorio con permisos restringidos)



2. Inicializar los ficheros de números de serie de certificados emitidos y revocados (contienen el siguiente número de serie en hexadecimal a utilizar). Crear el fichero índice (vacío) para la base de datos

   - Para crear un fichero vacío:

     `# touch nombre-fichero`

   - Para inicializar un fichero con un valor hexadecimal aleatorio:

     `# openssl rand -hex 16 > nombre-fichero`

[3] Otras opciones: Certificate Manager, YaST, Boulder ....
[4] La estructura y nombres adoptados, se indicarán en su fichero de configuración

3. Completar el fichero de configuración

   Asignar valores coherentes a todos los campos que figuren entre «... ...»

4. Generar claves de la *rootCA*, solicitud de certificado y certificado autofirmado[5]

   ```
   # openssl req -new [-nodes] -keyout clave-privada-rootCA.key \
         -out solicitud-rootCA.csr -config fichero-config-rootCA.cnf
   # openssl ca -selfsign \
         -in solicitud-rootCA.csr -out certificado-rootCA.crt \
         -config fichero-config-rootCA.cnf -extensions ca_ext
   ```

5. Generar una CRL vacía para una CA

   ```
   # openssl ca -gencrl -out crl-CA.pem \
         -config fichero-config-CA.cnf
   ```

6. Generar claves de la CA subordinada y solicitud de certificado. Emitir certificado firmado por la *rootCA*

   ```
   # openssl req -new [-nodes] -keyout clave-privada-subCA.key \
         -out solicitud-subCA.csr -config fichero-config-subCA.cnf
   # openssl ca \
         -in solicitud-subCA.csr -out certificado-subCA.crt \
         -config fichero-config-rootCA.cnf -extensions subca_ext
   ```

Emisión de un certificado por la CA subordinada

1. Generar claves y solicitud de certificado

   ```
   # openssl req -new [-nodes] -out solicitud-usuario.csr \
         -keyout clave-privada-usuario.key \
         [-subj datos-solicitud]
         [-addext exts-solicitud]
   ```

2. —— *AUTORIDAD DE REGISTRO* —— Validar información aportada por el usuario

3. Emitir el certificado

   ```
   # openssl ca -in solicitud-usuario.csr -out certificado-usuario.crt \
         -config fichero-config-subCA.cnf -extensions {server_ext, user_ext,
   ocsp_ext}
   ```

[5]Sustituir los nombres en cursiva por el **path completo** al fichero correspondiente

Revocación de un certificado

```
# openssl ca -revoke certificado-a-revocar.crt \
      -crl_reason keyCompromise -config fichero-config-CA.cnf

   reason: {keyCompromise, affiliationChanged, unspecified,
      CACompromise, superseded, removeFromCRL,...}
En función de las políticas de la PKI, actualizar CRL
```

Validación de un certificado de usuario (*OCSP responder)*

- Generar claves, solicitud de certificado y certificado para que el servidor OCSP de la subCA[6] pueda firmar sus respuestas.
- Poner en marcha un servidor OCSP (*responder*) con `OpenSSL`. Utiliza la base de datos creada por la *subCA*, y espera las peticiones de los clientes en el puerto indicado

  ```
  [# rm log-subCA.txt ]
  # openssl ocsp -port num_puerto -index indice_base_datos \
        -CA certificado-subCA.crt -rsigner certificado-subCA-ocsp.crt
  \
        -rkey clave-privada-subCA-ocsp.key -text -out log-subCA.txt
  ```

- Preguntar al servidor OCSP (actuando como cliente OCSP)

  ```
  # cat subCA.crt rootCA.crt > chain-certif.crt
  # openssl ocsp -CAfile chain-certif.crt \
        -issuer certificado-subCA.crt \
        -url http://oscp_servidor:num_puerto \
        -cert certificado-a-verificar -resp_text
        [-header "HOST"="url_oscp_servidor"]
  ```

Exportar certificado y clave privada

- Exportar el certificado y la clave privada de un usuario a un fichero con formato PKCS#12 (se establece una contraseña de cifrado a conocer por CA y usuario)

  ```
  openssl pkcs12 -export -out fichero.p12 -in certificado.crt
              -inkey clave-privada.key
  ```

[6] Si fuese necesario, se aplicarían los mismos pasos para poner en marcha el *OCSP Responder* para la *rootCA*.

- Extraer certificado y clave [7]

  `openssl pkcs12 -in` *`fichero`*`.p12 -out` *`certificado`*`.crt -nokeys`

  `openssl pkcs12 -in` *`fichero`*`.p12 -out` *`clave-privada`*`.key -nocerts`

  `openssl pkcs12 -info -in` *`fichero`*`.p12`

Opciones

- Para obtener una solicitud de certificado de forma no interactiva, se pasan todos los datos solicitados por medio de la opción `-subj` *"/campo0=valor0/campo1=valor1/.../campoN=valorN/"*. Por ejemplo:

  `-subj "/C=`*`ES`*`/ST=`*`GI`*`/L=`*`SS`*`/O=`*`SRDSI`*`/OU=`*`LABS-SRDSI`*`/`

  `CN=`*`ocsp.srdsi.lab`*`/emailAddress=`*`webmaster@srdsi.lab`*`/"`

- Generar certificados SAN (*subjectAltName*):

  - En la solicitud del certificado incluir la opción

    `-addext "subjectAltName = DNS:www.`*`srdsi.lab`*`, DNS:`*`srdsi.lab`*`"`

  - Comprobar que en el fichero de configuración está definida "`copy_extensions = copy`" para que las extensiones en la CSR se incluyan en el certificado

- Crear una cadena de certificados:

  `# cat` *`subCA.crt rootCA.crt > chain-certif.crt`*

- El formato de los ficheros generados por OpenSSL es PEM. Son ficheros de texto, pero no son legibles. Para comprobar el contenido de estos ficheros:

  `# openssl {req, x509, rsa, crl} -in` *`filename.pem`* `[-noout] -text`

- Algunos ficheros deben distribuirse en formato DER (formato binario) como, por ejemplo, las CRLs. Para cambiar de formato PEM a DER:

  `# openssl crl -inform PEM -in` *`fichero.pem`* `\`

  `-outform DER -out` *`fichero.der`*

Referencias

- «*Openssl cookbook*» I. Ristic (2ed, 2022)
- `http://pki-tutorial.readthedocs.org/en/latest/index.html`
- `man openssl ca`
- `man x509v3_config`

[7] Los navegadores permiten importar directamente el fichero .p12

```
### ca-root_openssl.cnf ###
[default]
name          = …nombre-CAroot…
domain_suffix = …nombre_dominio…
aia_url       = http://$name.$domain_suffix/$name.crt
crl_url       = http://$name.$domain_suffix/$name.crl
ocsp_url      = http://ocsp.$domain_suffix:…num_puerto…
default_ca    = CA_default   # Sección por defecto para CA
name_opt      = utf8,esc_ctrl,multiline,lname,align

[ca_dn]
countryName             = …nombre_país (2 letras)…
stateOrProvinceName     = …nombre_provincia (2 letras)…
localityName            = …nombre_localidad…
organizationName        = …nombre_organización…
organizationalUnitName  = …nombre_departamento…
commonName              = …nombre_identificador_certificado…


[CA_default]
home             = …directorio_principal_estructura…
db               = …directorio_base_datos…
privado          = …directorio_datos_privados…
database         = $home/$db/…fich_indice_db.txt…
serial           = $home/$db/…fich_num_serie_certificados.srl…
crlnumber        = $home/$db/…fich_num_serie_crl.srl…
certificate      = $home/$name.crt
private_key      = $home/$privado/$name.key
crl              = $home/$db/$name.crl           # (DER format)
RANDFILE         = $home/$privado/.rand
new_certs_dir    = $home/…directorio_nuevos_certificados…
unique_subject   = no
copy_extensions  = none
default_days     = 3650
default_crl_days = 300
default_md       = sha256
policy           = policy_c_o_match

# Política para la CA
[policy_c_o_match]
countryName             = match
stateOrProvinceName     = optional
organizationName        = match
organizationalUnitName  = optional
commonName              = supplied
emailAddress            = optional

[req]
default_bits       = 2048                     # 4096
encrypt_key        = yes
default_md         = sha256
utf8               = yes
string_mask        = utf8only
prompt             = no
distinguished_name = ca_dn

[ca_ext]
basicConstraints     = critical,CA:true
keyUsage             = critical,keyCertSign,cRLSign
subjectKeyIdentifier = hash

[crl_info]
URI.0              = $crl_url
```

```
[issuer_info]
caIssuers;URI.0    = $aia_url
OCSP;URI.0         = $ocsp_url

[ocsp_ext]
authorityKeyIdentifier  = keyid:always
basicConstraints        = critical,CA:false
extendedKeyUsage        = OCSPSigning
keyUsage                = critical,digitalSignature
subjectKeyIdentifier    = hash

[subca_ext]
authorityInfoAccess     = @issuer_info
authorityKeyIdentifier  = keyid:always
basicConstraints        = critical,CA:true,pathlen:0
crlDistributionPoints   = @crl_info
extendedKeyUsage        = clientAuth,serverAuth
keyUsage                = critical,keyCertSign,cRLSign
subjectKeyIdentifier    = hash
```

```
### ca-sub_openssl.cnf ###
[default]
name          = …nombre-SubCA…
domain_suffix = …nombre_dominio…
aia_url       = http://$name.$domain_suffix/$name.crt
crl_url       = http://$name.$domain_suffix/$name.crl
ocsp_url      = http://ocsp.$domain_suffix:…num_puerto…
default_ca    = CA_default   # Sección por defecto para CA
name_opt      = utf8,esc_ctrl,multiline,lname,align

[ca_dn]
countryName             = …nombre_país (2 letras)…
stateOrProvinceName     = …nombre_provincia (2 letras)…
localityName            = …nombre_localidad…
organizationName        = …nombre_organización…
organizationalUnitName  = …nombre_departamento…
commonName              = …nombre_identificador_certificado…


[CA_default]
home             = …directorio_principal_estructura…
db               = …directorio_base_datos…
privado          = …directorio_datos_privados…
database         = $home/$db/…fich_indice_db.txt…
serial           = $home/$db/…fich_num_serie_certificados.srl…
crlnumber        = $home/$db/…fich_num_serie_crl.srl…
certificate      = $home/$name.crt
private_key      = $home/$privado/$name.key
crl              = $home/$db/$name.crl           # (DER format)
RANDFILE         = $home/$privado/.rand
new_certs_dir    = $home/…directorio_nuevos_certificados…
unique_subject   = no
copy_extensions  = copy
default_days     = 3650
default_crl_days = 300
default_md       = sha256
policy           = policy_c_o_match

# Política para la CA
[policy_c_o_match]
countryName             = match
stateOrProvinceName     = optional
organizationName        = match
organizationalUnitName  = optional
commonName              = supplied
emailAddress            = supplied

[req]
default_bits       = 2048                     # 4096
encrypt_key        = yes
default_md         = sha256
utf8               = yes
string_mask        = utf8only
prompt             = no
distinguished_name = ca_dn


[crl_info]
URI.0     = $crl_url

[issuer_info]
caIssuers;URI.0    = $aia_url
OCSP;URI.0         = $ocsp_url
```

```
[ocsp_ext]
authorityKeyIdentifier  = keyid:always
basicConstraints        = critical,CA:false
extendedKeyUsage        = OCSPSigning
keyUsage                = critical,digitalSignature
subjectKeyIdentifier    = hash

[server_ext]
authorityInfoAccess     = @issuer_info
authorityKeyIdentifier  = keyid:always
basicConstraints        = critical,CA:false
crlDistributionPoints   = @crl_info
extendedKeyUsage        = clientAuth,serverAuth
keyUsage                = critical,digitalSignature,keyEncipherment
subjectKeyIdentifier    = hash
subjectAltName           = email:copy

[user_ext]
authorityInfoAccess     = @issuer_info
authorityKeyIdentifier  = keyid:always
basicConstraints        = critical,CA:false
crlDistributionPoints   = @crl_info
extendedKeyUsage        = clientAuth
keyUsage                = critical,digitalSignature
subjectKeyIdentifier    = hash
```